# Comprehensive Cloud Security Engineering

**Final Project**: Cyber 290

**Group:** Tyler Tenlen, Slam Laqtib

---

# Table of Contents

1. **Overview**
2. **Overall Execution Plan**
    - Foundation: Identity and Access Control
    - Data Protection (Cryptography)
    - Core Network Segmentation
    - Secure Application Deployment
    - Governance and Audit Trail
    - Automated Threat Response
    - Advanced Access Control: Zero Trust
3. **Architecture — The Big Picture**
4. **Implementation — End-To-End**
    - 4.1. Identity Management and Privilege Control
        - 4.1.1. Prince of Least Privilege (PoLP)
        - 4.1.2. Users, Policies, and Cloudsplaining
    - 4.2. Data Security and Encryption
        - 4.2.1. Data-at-Rest Protection
        - 4.2.2. Key Management — SSE-KMS and Key Policies
    - 4.4. Secure Networking Design
        - 4.4.1. Defense-in-Depth Networking
        - 4.4.2. VPC, Subnets, Routing, and Access Controls
    - 4.4. Secure Application Deployment and Hardening
        - 4.4.1. Secure Deployment and Supply Chain
        - 4.4.2. IaC, TLS, IAM Roles, and Trivy Scanning
    - 4.5. Operational Visibility and Auditing
        - 4.5.1. Audit Trail and Security Assessment
        - 4.5.2. CloudTrail Deployment and Prowler Audit
    - 4.6. Automated Threat Detection and Response
        - 4.6.1. Serverless Remediation Pipeline
        - 4.6.2. GuardDuty, EventBridge, and Lambda Automation
    - 4.7. Advanced Access Control: Zero Trust
5. **Reflection & Lessons Learned**
6. **Appendix**
    - Project Technologies, Security Tools and Their Roles Summary

# 1. Overview

This hands-on project documents a series of comprehensive security architectures conducted within an Amazon Web Services (AWS) environment, focusing on foundational principles and advanced techniques in cloud security engineering. The project's core objective was to establish a multi-layered, Defense-in-Depth security posture, moving beyond theoretical knowledge to the practical, hands-on implementation of an end-to -end, robust, and secure application ecosystem.

Our work explores critical subjects, including: the Principle of Least Privilege (PoLP) applied across Identity and Networking; the implementation of data-at-rest encryption using AWS Key Management Service (KMS); data-in-transit encryption using TLS; establishing secure network segmentation via a Virtual Private Cloud (VPC); employing Infrastructure as Code (IaC) with Terraform for repeatable, version controlled deployments; and finally, designing automated threat detection and response workflows. By simulating real-world scenarios, including intentional misconfigurations and auditing against industry best practices, this project demonstrates a complete lifecycle approach to building, securing, and continuously monitoring cloud infrastructure.

---

# 2. Overall Execution Plan

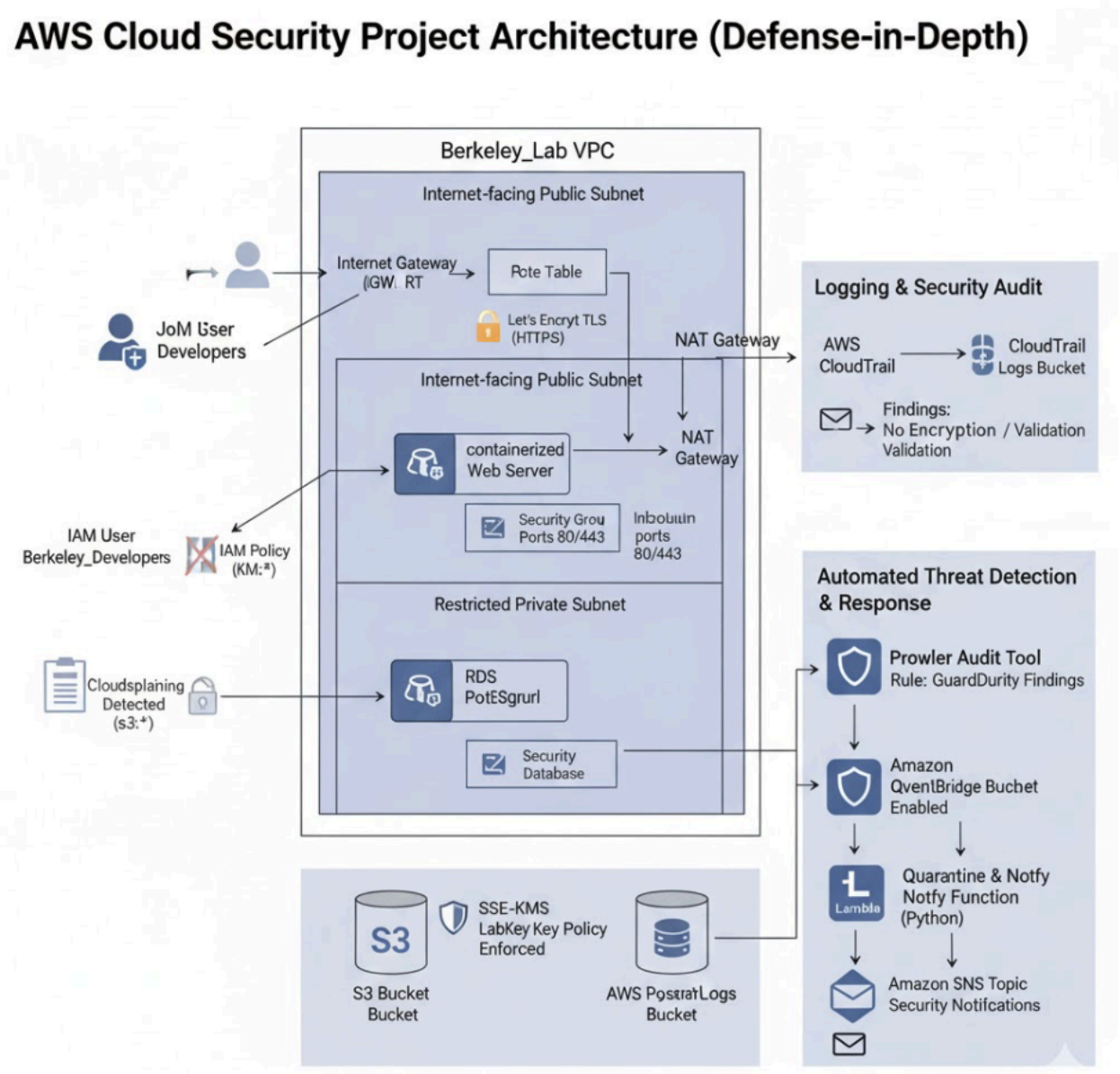
The project was executed through a systematic, phased approach, building each security control as a solid foundation to achieve comprehensive security coverage.

| **Phase** | **Description** | **Key Technologies & Principles** |
|---|---|---|
| **Foundation: Identity and Access Control** | Configure and audit Identity and Access Management (IAM) to demonstrate the Principle of Least Privilege (PoLP) and identify the security risks of over-permissioning. | AWS IAM, PoLP, Cloudsplaining |
| **Data Protection** | Implement server-side encryption to protect sensitive data at rest using a customer-managed key and enforce | AWS KMS, SSE-KMS, IAM Key Policies |

| | | |
|---|---|---|
| | layered access control based on both bucket and key policies. | |
| **Core Network Segmentation** | Design and deploy a secure Virtual Private Cloud (VPC) with segregated public and private subnets, controlled routing, and a zero exposure instance management model. | AWS VPC, IGW, NAT Gateway, Security Groups, AWS Systems Manager |
| **Secure Application Deployment** | Use Infrastructure as Code (IaC) to deploy a containerized web application, secure data-in-transit with TLS, and implement supply chain security checks. | Terraform, TLS (Let's Encrypt), IAM Roles, Trivy |
| **Governance and Audit Trail** | Enable and audit AWS CloudTrail to establish a reliable audit trail for governance, compliance, and incident response, validating the configuration against best practices. | AWS CloudTrail, Terraform, Prowler |
| **Automated Threat Response** | Build a serverless, end-to-end pipeline for automated threat detection and remediation, drastically reducing Mean Time to Respond (MTTR). | AWS GuardDuty, EventBridge, Lambda, SNS |
| **Advanced Access Control: Zero Trust** | Implement a modern Zero Trust model for access control, using a policy engine to enforce access based on verified user identity and device posture. | Open Policy Agent (OPA), Rego, KICS |

# 3. Architecture — The Big Picture

The following comprehensive diagram shows the essential components of this project, providing visual clarity for the complex architectures and workflows implemented across our Cyber 290 class Labs.



- **Network Architecture:** A visual representation of the VPC, Public/Private Subnets, IGW, and NAT Gateway that clearly shows the separation of the Web Server and Database EC2 instances.
- **Application Architecture:** A visual representation of coordinated defense illustrating the EC2 instance, Security Group, IAM Role, DNS, and TLS/Certbot flow to show data encryption and authentication.
- **Automated Response Flow:** A visualization of the serverless event pipeline: GuardDuty Finding → EventBridge Rule → Lambda Function → Quarantine Bucket + SNS Notification, illustrating the MTTR reduction.
- **Logging and Auditing Flow:** A visual representation showing the flow of API Calls → CloudTrail → S3 Log Bucket, demonstrating the audit trail infrastructure.

---

# 4. Implementation — End-To-End

The implementation section details the specific steps taken across each security subproject, providing a technical overview of the controls implemented and the principles demonstrated.

## 4.1. Identity Management and Privilege Control

### 4.1.1. Prince of Least Privilege (PoLP)

This workstream established foundational security by focusing on the Principle of Least Privilege (PoLP). We created the user *John_Developer* within the *Berkeley_Developers* group. The correct implementation of PoLP was demonstrated by assigning the *developer-s3-app-logging-prod* policy, which was narrowly scoped to grant only ***List, Get, and Put*** S3 actions on a single, specific application log bucket.

### 4.1.2. Users, Policies, and Cloudsplaining

We then intentionally violated PoLP by creating an overly permissive policy (*s3:* on **), simulating a security flaw that could lead to lateral access to sensitive financial data. The open-source auditing tool **Cloudsplaining** was successfully utilized to detect this high risk misconfiguration, leading to the remediation of the policy to enforce strict access controls.

## 4.2. Data Security and Encryption

### 4.2.1. Data-at-Rest Protection

The objective was to ensure data-at-rest protection through layered access control, making the data accessible only by users who had both the necessary S3 permissions and the authorization to use the specific AWS Key Management Service (KMS) key.

### 4.2.2. Key Management — SSE-KMS and Key Policies

We created a symmetric customer-managed key named *LabKey* and enforced Server-Side Encryption (SSE-KMS) as the default for a target S3 bucket. Crucially, the Key Policy for *LabKey* was configured to explicitly grant *John_Developer* key usage permissions. This design ensures that access to the encrypted data is governed by two independent controls: the S3 Bucket Policy (resource permissions) and the KMS Key Policy (cryptographic permissions).

## 4.4. Secure Networking Design

### 4.4.1. Defense-in-Depth Networking

A robust network architecture was established using a Virtual Private Cloud (VPC), segmented into a Public Subnet (for the internet facing web server) and a Private Subnet (for the sensitive/internal database), ensuring strong segmentation and controlled routing.

### 4.4.2. VPC, Subnets, Routing, and Access Controls

Routing was tightly controlled: the Private Subnet routed all outbound traffic through a NAT Gateway, blocking all direct inbound access. Network Security Groups were applied following PoLP. A key security enhancement was the use of an IAM Role with AWS Systems Manager (SSM) to securely manage the private EC2 instance, implementing a "zero-exposure" security model that eliminated the need for a traditional Bastion Host.

## 4.4. Secure Application Deployment and Hardening

### 4.4.1. Secure Deployment and Supply Chain

A containerized Apache web application was deployed on an EC2 instance using Terraform for full Infrastructure as Code (IaC) management. The focus was on securing the deployment with proper network access controls, to secure data-in-transit, and to ensure software supply chain integrity.

### 4.4.2. IaC, TLS, IAM Roles, and Trivy Scanning

Data-in-transit was secured by configuring TLS/HTTPS using a **Let's Encrypt** certificate. Security of the application host was enhanced by assigning an IAM Role to the EC2 instance, which allowed it to request temporary, automatically rotated

credentials. The **Trivy** tool was used to scan the container image for known supply chain vulnerabilities.

## 4.5. Operational Visibility and Auditing

### 4.5.1. Audit Trail and Security Assessment

The primary objective was to deploy, verify, and security audit AWS CloudTrail, the foundational service for governance, compliance, and incident response, to ensure the establishment of a reliable audit trail for all API activities.

### 4.5.2. CloudTrail Deployment and Prowler Audit

The CloudTrail service was deployed using Terraform. The deployed configuration was then subjected to a security audit using the **Prowler** tool. The audit revealed that the default configuration lacked essential security hardening, specifically failing to enable log file validation and KMS encryption, necessitating deliberate configuration changes beyond the typical out-of-the-box settings.

## 4.6. Automated Threat Detection and Response

### 4.6.1. Serverless Remediation Pipeline

This assignment demonstrated the end-to-end deployment of an automated, serverless malware remediation pipeline, aiming to deliver a low-cost, automated threat detection and response capability.

### 4.6.2. GuardDuty, EventBridge, and Lambda Automation

AWS GuardDuty was enabled with the S3 Malware Protection feature. The event-driven architecture was built using Amazon EventBridge to listen for malware findings, which automatically triggered a Python AWS Lambda function. This function performed the automated two-part remediation: 1) copying the malicious object to a quarantine bucket, and 2) deleting the object from the source, drastically reducing the Mean Time to Respond (MTTR).

## 4.7. Advanced Access Control: Zero Trust

The final workstream implemented a modern Zero Trust access model, requiring explicit verification before granting access. Access control logic was defined in a "default deny" Open Policy Agent (OPA) policy using the Rego language. The policy enforced that access was permitted only when a user had the correct least-privilege role *and* a verified device posture.

---

# 5. Reflection & Lessons Learned

The cumulative experience we have gained across these security workstreams reinforced several critical principles and provided valuable operational lessons necessary for any professional cloud security engineering.

## Core Security Principles in Practice

The Principle of Least Privilege (PoLP) was the most central and profound theme. We learned that PoLP implementation is a continuous auditing effort, rather than a one-off configuration. The intentional violation of PoLP confirmed that its failure creates a catastrophic security risk of lateral movement to sensitive data, making continuous monitoring (like that performed by Cloudsplaining) essential for enforcement. Furthermore, the entire project reinforced the concept of Defense-in-Depth, proving that a secure architecture requires independent, overlapping controls across identity, network, cryptography, and application layers.

## Operational and Automation Complexity

The use of Infrastructure as Code (IaC) with Terraform was critical for ensuring consistency and repeatability. This was complemented by the successful deployment of the GuardDuty-Lambda pipeline, which demonstrated that serverless automation can drastically reduce the Mean Time to Respond (MTTR) from hours to seconds. This automation, however, highlighted a key operational lesson: the true complexity lies in the careful, layered configuration of permissions. The smallest misconfigured policy can halt the entire workflow, requiring exhaustive, end-to-end testing of permissions. The network workstream also showed the clear benefit of a **"**zero-exposure" approach by using IAM roles with AWS Systems Manager (SSM) for private resource access, eliminating the need for security compromising inbound firewall rules.

## Visibility Gaps and Configuration Pitfalls

Our work with CloudTrail demonstrated that default cloud configurations are often insufficient for true security and compliance. The audit with Prowler revealed that achieving a comprehensive audit capability requires deliberate configuration *beyond* the out-of-the-box settings, specifically enabling log file validation and KMS encryption. A significant and costly lesson was learned when using AWS IAM Access Analyzer: enabling the feature triggered immediate billing based on the number of resources analyzed per month. This emphasized the non-obvious and non-linear cost models in the cloud, underscoring the critical need to fully vet pricing documentation before activating any new services.

## Advancing Access Control

The Zero Trust effort confirmed that an effective architecture requires both verified identity and verified device posture. While effective, the implemented model is an early stage; for production maturity, the system should integrate dynamic verification using tools like GuardDuty and IAM Access Analyzer to ensure policies remain robust and adapt to real-time user behavior, ensuring continuous verification beyond a static rule set.

# Appendix

### Project Technologies, Security Tools and Their Roles Summary

This list summarizes the core technologies and security tools used across the project and defines their specific role in implementing the Defense-in-Depth security posture.

| Tool / Service | Role in Project |
|---|---|
| **AWS IAM (Identity and Access Management)** | Core service for managing users, groups, and permissions. Used to implement and test the Principle of Least Privilege (PoLP) via tightly scoped policies and roles. |
| **AWS KMS (Key Management Service)** | Created and managed symmetric customer-managed keys (LabKey). Enforced layered access control for data-at-rest protection (SSE-KMS). |
| **AWS S3 (Simple Storage Service)** | Storage target for application logs and sensitive data. Enforced SSE-KMS encryption and served as the target for automated malware scanning/quarantine. |
| **AWS VPC (Virtual Private Cloud)** | Core network service used to establish a secure, segmented architecture with public/private subnets, enforcing Defense-in-Depth. |
| **AWS Systems Manager (SSM)** | Enabled secure access to private EC2 instances via IAM roles zero-exposure management with no bastion host required. |
| **AWS CloudTrail** | Deployed via IaC to create a consistent audit trail for governance, compliance, and incident response by logging all API activity. |

| | |
|---|---|
| **AWS GuardDuty** | Threat detection service with S3 Malware Protection. Scanned uploaded files for malware and triggered the automated response workflow. |
| **Amazon EventBridge** | Event-driven service configured to trigger the system by listening for GuardDuty malware findings. |
| **AWS Lambda** | Serverless Python function executing automated response logic: quarantining malicious files and sending alerts. |
| **Amazon SNS (Simple Notification Service)** | Sent detailed notifications (via email) to the security team after automated remediation. |
| **Terraform** | IaC tool used to provision the full cloud environment (VPC, IAM, GuardDuty, Lambda, CloudTrail, etc.) consistently and repeatedly. |
| **Cloudsplaining** | Audited IAM to detect overly permissive policies (e.g., s3:* on *) that violate PoLP. |
| **Prowler** | Audited CloudTrail against AWS best practices, identifying missing hardening requirements such as log file validation. |
| **Let's Encrypt / Certbot** | Issued and automatically renewed TLS certificates to ensure encrypted data-in-transit. |
| **Trivy** | Scanned the application's container image for supply-chain vulnerabilities to ensure software integrity. |
| **Open Policy Agent (OPA)** | Policy engine enforcing Zero Trust access controls using identity and device posture checks. |
| **Rego** | Policy language for OPA, defining fine-grained access rules for the Zero Trust implementation. |
| **KICS (Keeping Infrastructure as Code Secure)** | Scanned Terraform code for vulnerabilities and misconfigurations before deployment. |